

02. 分治与递归式

算法设计与分析

分治思想

- ▶ 分 (Divide): 将原问题分解为 k 个规模较小的子问题
- ▶ 治 (Conquer): 递归地解决各子问题（子问题足够小时直接求解）
- ▶ 合 (Combine): 将子问题的解合并为原问题的解
- ▶ 适用条件: 子问题相互独立、与原问题同构、可合并

分治算法框架

```
function DivideAndConquer(P):  
    if  $|P| \leq n_0$ : return DirectSolve(P)  
    将 P 分解为  $P_1, P_2, \dots, P_k$   
    for  $i = 1$  to  $k$ :  
         $y_i = \text{DivideAndConquer}(P_i)$   
    return Merge( $y_1, y_2, \dots, y_k$ )
```

经典案例：归并排序

- ▶ 分：将数组从中间分为左右两半
- ▶ 治：递归对左右两半分别排序
- ▶ 合：将两个有序数组合并为一个有序数组
- ▶ 时间复杂度： $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$
- ▶ 空间复杂度： $O(n)$ ，需要额外数组用于合并

经典案例：快速排序

- ▶ 分：选择基准 `pivot`，将数组分为 `<pivot` 和 `>pivot` 两部分
- ▶ 治：递归对左右分区排序
- ▶ 合：无需额外合并，分区完成即有序
- ▶ 平均时间： $T(n) = 2T(n/2) + O(n) \rightarrow \Theta(n \log n)$
- ▶ 最坏时间：已排序数组， $O(n^2)$ ，可通过随机 `pivot` 避免

经典案例：二分查找

- ▶ 前提：有序数组，目标值与中间元素比较
- ▶ 分：比较中间元素，确定目标在左半还是右半
- ▶ 治：在确定的半区递归查找
- ▶ 合：直接返回结果，无需合并
- ▶ 时间复杂度： $T(n) = T(n/2) + O(1) \rightarrow \Theta(\log n)$

经典案例：大整数乘法

- ▶ 问题：两个 n 位大整数相乘，朴素算法 $O(n^2)$
- ▶ Karatsuba 算法：将 n 位数拆分为高位和低位
- ▶ $X = a \cdot 10^{(n/2)} + b$, $Y = c \cdot 10^{(n/2)} + d$
- ▶ $XY = ac \cdot 10^n + (ad+bc) \cdot 10^{(n/2)} + bd$
- ▶ 优化： $ad+bc = (a+b)(c+d) - ac - bd$ ，仅需 3 次乘法
- ▶ 复杂度： $T(n) = 3T(n/2) + O(n) \rightarrow \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$

递归式定义

- ▶ 描述分治算法运行时间的递推方程
- ▶ 一般形式: $T(n) = a \cdot T(n/b) + f(n)$
- ▶ a : 子问题数量 (每次递归产生多少个子问题)
- ▶ n/b : 每个子问题的规模 (假设均分)
- ▶ $f(n)$: 分解 (Divide) 和合并 (Combine) 步骤的总代价

求解方法：代入法

- ▶ 步骤1：猜测解的形式，如 $T(n) = O(n \log n)$
- ▶ 步骤2：用数学归纳法证明 $T(n) \leq c \cdot n \cdot \log n$
- ▶ 步骤3：验证归纳基例和归纳步骤
- ▶ 技巧：引入低阶项辅助证明，如 $T(n) \leq c \cdot n \cdot \log n - d \cdot n$
- ▶ 适用：有明确猜测时，或验证递归树得到的解

求解方法：递归树法

- ▶ 画出递归调用树，根节点代价为 $f(n)$
- ▶ 每个内部节点有 a 个子节点，代价为 $f(n/b^i)$
- ▶ 计算树的高度： $h = \log_b n$
- ▶ 计算每层总代价，然后对所有层求和
- ▶ 示例： $T(n) = 3T(n/4) + cn^2$ ，每层代价呈几何级数递减

主定理 (Master Theorem)

▸ 形式: $T(n) = aT(n/b) + f(n)$, $a \geq 1, b > 1$

▸ 比较 $f(n)$ 与 $n^{\log_b a}$ 的渐近关系

【情况1】 $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$

【情况2】 $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$

【情况3】 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 且正则条件 $\rightarrow T(n) = \Theta(f(n))$

主定理应用实例

- ▶ 归并排序: $T(n)=2T(n/2)+O(n)$, $a=2, b=2, \log_2 2=1$
→ $n^1 = n$, $f(n)=\Theta(n)$, 符合情况2 → $\Theta(n \log n)$
- ▶ 二分查找: $T(n)=T(n/2)+O(1)$, $a=1, b=2, \log_2 1=0$
→ $n^0=1$, $f(n)=\Theta(1)$, 符合情况2 → $\Theta(\log n)$
- ▶ Strassen: $T(n)=7T(n/2)+O(n^2)$, $a=7, b=2, \log_2 7 \approx 2.81$
→ $n^{2.81}$, $f(n)=O(n^2)$, 符合情况1 → $\Theta(n^{2.81})$

分治与递归的注意事项

- ▶ 递归深度：过深导致栈溢出，可改为迭代或尾递归优化
- ▶ 子问题重叠：若子问题不独立，考虑动态规划替代
- ▶ 小规模优化： n 较小时改用直接算法（如插入排序），减少递归开销
- ▶ 均分假设：实际中子问题可能不均等，需调整分析